INSTITUTO FEDERAL DO ESPÍRITO SANTO
MESTRADO PROFISSIONAL EM COMPUTAÇÃO APLICADA

FLÁVIO MIOZZI BATISTA

LABORATÓRIO 01: LINGUAGENS REGULARES

Serra
2026

FLÁVIO MIOZZI BATISTA

LABORATÓRIO 01: LINGUAGENS REGULARES

Relatório de atividade de laboratório apresentado ao Mestrado Profissional em Computação Aplicada do Instituto Federal do Espírito Santo, como parte das atividades da disciplina Teoria da Computação, semestre 2026/1.

Resumo. Este trabalho descreve a implementação, em Haskell, de algoritmos clássicos de teoria dos autômatos: (1) conversão de NFAε para DFA mínimo via remoção de ε-transições, construção de subconjuntos e minimização por Hopcroft; e (2) Construção de Thompson para converter expressões regulares em NFAε, seguida do mesmo pipeline de conversão. São documentados seis autômatos e cinco expressões regulares de exemplo, com entrada e saída em formato YAML. A Parte 3 apresenta soluções da plataforma *Regex Crossword*.

Palavras-chave: Autômatos finitos. Expressões regulares. Construção de Thompson. Algoritmo de Hopcroft. Haskell.

Orientador: Prof. Jefferson Oliveira Andrade

Serra
2026

Lista de ilustrações

Figura 1 – NFAε `nfae_simple`: $L = 0^*1$. 7
Figura 2 – NFA intermediário de `nfae_simple` (sem transições ε). 7
Figura 3 – DFA mínimo de `nfae_simple`: $L = 0^*1$. 8
Figura 4 – NFAε `nfae_02`: $L = \{a, b\}$. 8
Figura 5 – DFA mínimo de `nfae_02`: $L = \{a, b\}$. 8
Figura 6 – NFAε `nfae_04`: $L = \{ab\}$. 8
Figura 7 – DFA mínimo de `nfae_04`: $L = \{ab\}$. 9
Figura 8 – NFAε `nfae_06` — construção de Thompson para $(a|b)^*$. 9
Figura 9 – DFA mínimo de `nfae_06`: $L = (a|b)^*$ (estado único, inicial e final). . . 9
Figura 10 – NFAε `nfae_08`: $L = (0|1)^*01$. 10
Figura 11 – DFA mínimo de `nfae_08`: $L = (0|1)^*01$. 10
Figura 12 – NFAε `nfae_10` (13 estados, $\Sigma = \{a, b, c\}$). 11
Figura 13 – DFA mínimo de `nfae_10` (8 estados, $\Sigma = \{a, b, c\}$). Estados finais: D_6 e D_7. Transições completas na Tabela 3. 12
Figura 14 – DFA mínimo para `ab*`: $S_0 = \{q_0\}$, $S_1 = \{q_3, q_4, q_5\}$. 13
Figura 15 – DFA mínimo para `a*b+`: $S_0 = \{q_0\}$, $S_1 = \{q_7, q_8, q_9\}$. 13
Figura 16 – DFA mínimo para `(a|b)*abb`. 14
Figura 17 – DFA mínimo para `a?`: $S_0 = \{q_0\}$, $S_1 = \{q_1, q_3\}$. 14
Figura 18 – DFA mínimo para `(a|b)*c(a+b|ba?)*`. R_0=$\{q_0\}$; R_1, R_2 finais; R_3 não-final. 15
Figura 19 – Cinco desafios da seção *Beginner* concluídos: Beatles, Naughty, Ghost, Symbolism e Airstrip One. 17
Figura 20 – Desafio *Beatles* — solução: HELP. 18
Figura 21 – Desafio *Naughty* — solução: BOBE. 19
Figura 22 – Desafio *Ghost* — solução: OOOO. 20
Figura 23 – Desafio *Symbolism* — solução: `**//` 20
Figura 24 – Desafio *Airstrip One* — solução: 1984. 21
Figura 25 – Quebra-cabeça FLAVIO_HOUSE_IFES — grade retangular 5×6. . . . 22
Figura 26 – Quebra-cabeça FLAVIO_MB_IFES — grade hexagonal, tamanho 10. 22

Sumário

1 INTRODUÇÃO 5

2 PARTE 1 — CONVERSÃO DE AUTÔMATOS FINITOS . . . 7

2.1 Visão Geral do Algoritmo 7

2.2 Exemplo 1 — `nfae_simple` ($L = 0^*1$) 7

2.3 Exemplo 2 — `nfae_02` ($L = \{a, b\}$) 8

2.4 Exemplo 3 — `nfae_04` ($L = \{ab\}$) 8

2.5 Exemplo 4 — `nfae_06` ($L = (a|b)^*$) 9

2.6 Exemplo 5 — `nfae_08` ($L = (0|1)^*01$) 9

2.7 Exemplo 6 — `nfae_10` 10

3 PARTE 2 — EXPRESSÕES REGULARES 13

3.1 Visão Geral — Construção de Thompson 13

3.2 Expressão 1 — `ab*` 13

3.3 Expressão 2 — `a*b+` 13

3.4 Expressão 3 — `(a|b)*abb` 14

3.5 Expressão 4 — `a?` 14

3.6 Expressão 5 — `(a|b)*c(a+b|ba?)*` 14

4 PARTE 3 — ATIVIDADE COMPLEMENTAR: REGEX CROSSWORD 17

4.1 Introdução 17

4.2 Desafios da Seção Beginner 17

4.2.1 Desafio 1 — Beatles 17

4.2.2 Desafio 2 — Naughty 18

4.2.3 Desafio 3 — Ghost 19

4.2.4 Desafio 4 — Symbolism 20

4.2.5 Desafio 5 — Airstrip One 20

4.3 Quebra-cabeças Criados pelo Aluno 21

4.3.1 Quebra-cabeça 1 — FLAVIO_HOUSE_IFES (Grade 5×6) 21

4.3.2 Quebra-cabeça 2 — FLAVIO_MB_IFES (Grade Hexagonal) . . 22

1 Introdução

Este relatório documenta as três partes do Laboratório 01 da disciplina Teoria da Computação do PPCOMPI/IFES.

A **Parte 1** trata da conversão de Autômatos Finitos Não-Determinísticos com transições ε (NFAε) para Autômatos Finitos Determinísticos Mínimos (DFA), por meio de três etapas: remoção de transições ε, construção de subconjuntos e minimização pelo algoritmo de Hopcroft. A implementação encontra-se em `Main.hs`.

A **Parte 2** aborda a construção de Thompson, que converte uma expressão regular em um NFAε, o qual é então convertido para DFA mínimo pelo mesmo pipeline da Parte 1. A implementação encontra-se em `Regex.hs`.

A **Parte 3** apresenta soluções para a plataforma *Regex Crossword*, que exercita a leitura e escrita de expressões regulares.

2 Parte 1 — Conversão de Autômatos Finitos

2.1 Visão Geral do Algoritmo

A conversão NFA$\varepsilon \to$ DFA mínimo implementada em `Main.hs` realiza três etapas sequenciais:

1. **Remoção de transições ε:** para cada estado q, computa-se o ε-fecho $\varepsilon^*(q)$ — o conjunto de estados alcançáveis por zero ou mais transições ε. As transições do novo NFA incorporam esses fechos.

2. **Construção de subconjuntos (*subset construction*):** cada estado do DFA representa um subconjunto de estados do NFA. As transições são expandidas iterativamente a partir do ε-fecho do estado inicial.

3. **Minimização por Hopcroft:** particiona os estados em classes de equivalência por distinguibilidade, colapsando estados indistinguíveis em um único representante.

2.2 Exemplo 1 — `nfae_simple` ($L = 0^*1$)

O autômato `nfae_simple` possui 3 estados sobre $\Sigma = \{0, 1\}$ e reconhece $L = 0^*1$. A transição ε de q_0 para q_1 permite ao autômato alcançar o estado de espera do símbolo 1 sem consumir entrada.

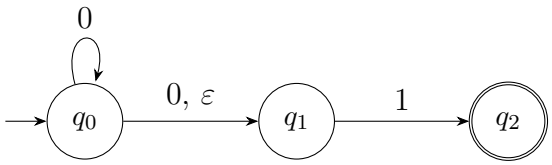


Figura 1 – NFAε `nfae_simple`: $L = 0^*1$.

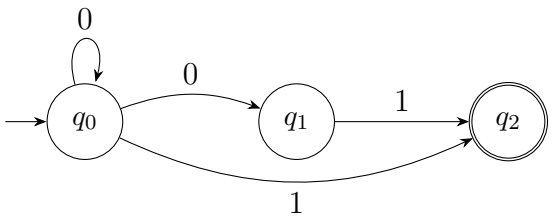


Figura 2 – NFA intermediário de `nfae_simple` (sem transições ε).

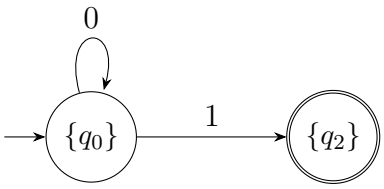


Figura 3 – DFA mínimo de `nfae_simple`: $L = 0^*1$.

2.3 Exemplo 2 — `nfae_02` ($L = \{a, b\}$)

O autômato `nfae_02` possui 5 estados e reconhece $L = \{a, b\}$ (exatamente uma letra). As transições ε de q_0 modelam a escolha não-determinística entre os ramos a e b.

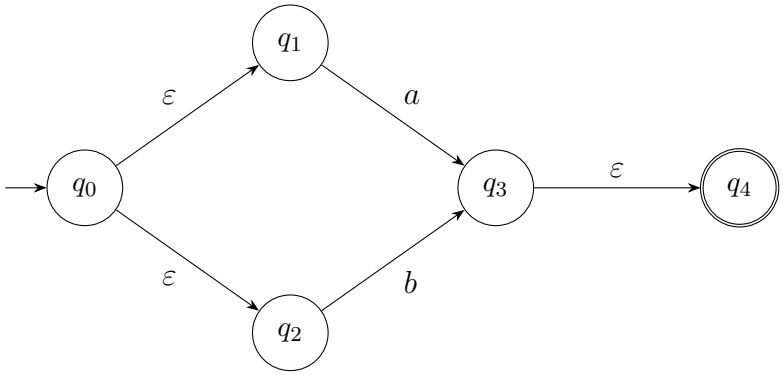


Figura 4 – NFAε `nfae_02`: $L = \{a, b\}$.

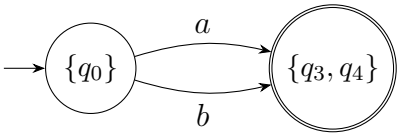


Figura 5 – DFA mínimo de `nfae_02`: $L = \{a, b\}$.

2.4 Exemplo 3 — `nfae_04` ($L = \{ab\}$)

O autômato `nfae_04` possui 4 estados e reconhece $L = \{ab\}$. A transição ε entre q_1 e q_2 conecta os dois estágios de leitura.

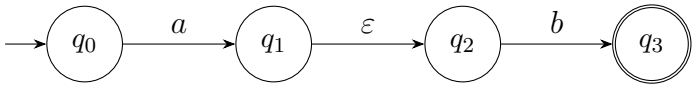


Figura 6 – NFAε `nfae_04`: $L = \{ab\}$.

Após a construção de subconjuntos, o estado $\{q_1, q_2\}$ surge porque $q_2 \in \varepsilon^*(q_1)$.

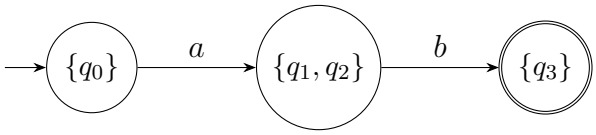


Figura 7 – DFA mínimo de `nfae_04`: $L = \{ab\}$.

2.5 Exemplo 4 — `nfae_06` ($L = (a|b)^*$)

O autômato `nfae_06` é a construção de Thompson para $(a|b)^*$, com 8 estados. O estado inicial q_0 possui transição ε direta para q_7 (aceitando a cadeia vazia) e para q_1 (ramo interno do Kleene). O arco $q_6 \to q_1$ implementa a repetição.

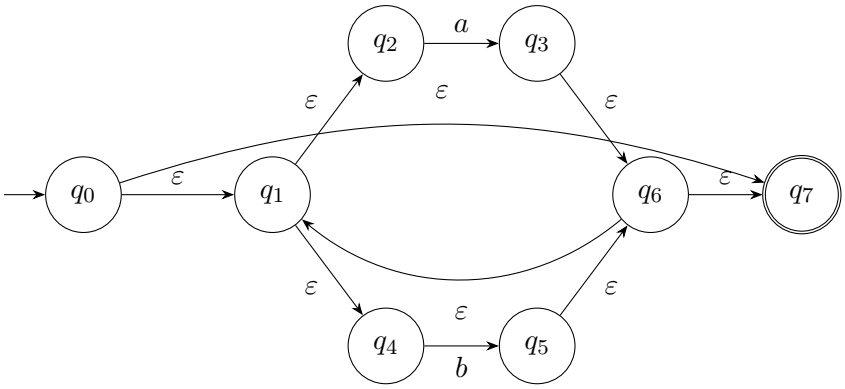


Figura 8 – NFAε `nfae_06` — construção de Thompson para $(a|b)^*$.

Como $(a|b)^*$ aceita qualquer cadeia sobre $\{a, b\}$, o DFA mínimo colapsa para um único estado, inicial e final:

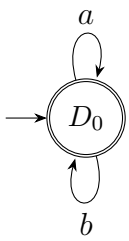


Figura 9 – DFA mínimo de `nfae_06`: $L = (a|b)^*$ (estado único, inicial e final).

2.6 Exemplo 5 — `nfae_08` ($L = (0|1)^*01$)

O autômato `nfae_08` possui 4 estados sobre $\Sigma = \{0, 1\}$ e reconhece $L = (0|1)^*01$ — todas as cadeias que terminam com o sufixo "01". O estado q_0 lê qualquer prefixo em $\{0, 1\}^*$; a transição ε para q_1 inicia o reconhecimento do sufixo.

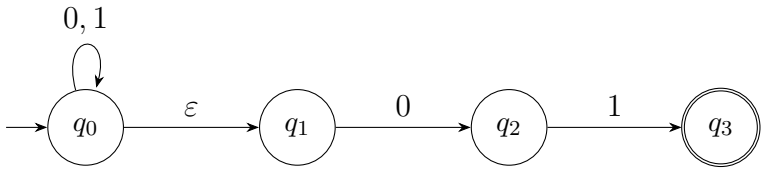


Figura 10 – NFAε `nfae_08`: $L = (0|1)^*01$.

A construção de subconjuntos seguida de minimização produz um DFA mínimo com 3 estados (Tabela 1). Os três estados codificam as situações relevantes ao longo da leitura: D_0 — nenhum prefixo útil para "01"; D_1 — o último caractere lido foi 0 (primeiro símbolo do sufixo); D_2 — os últimos dois caracteres foram "01" (aceito).

Tabela 1 – Tabela de transições do DFA mínimo de `nfae_08`.

Estado	**Subconjunto NFA**	0	1
$\rightarrow D_0$	$\{q_0\}$	D_1	D_0
D_1	$\{q_0, q_1, q_2\}$	D_1	D_2
D_2 *	$\{q_0, q_1, q_3\}$	D_1	D_0

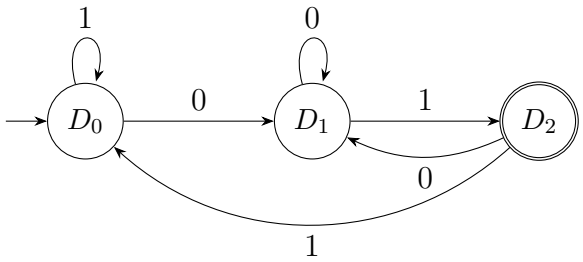


Figura 11 – DFA mínimo de `nfae_08`: $L = (0|1)^*01$.

2.7 Exemplo 6 — `nfae_10`

O autômato `nfae_10` possui 13 estados sobre $\Sigma = \{a, b, c\}$ e reconhece a linguagem L das cadeias que: (i) começam com ab ou cb; (ii) contêm ao menos um c intermediário; e (iii) terminam com ba ou cc.

O grafo de transições (Figura 12) estrutura-se em três segmentos: o par $(q_0 \rightarrow q_1 \rightarrow q_2,\ q_0 \rightarrow q_3 \rightarrow q_4) \rightarrow q_5$ codifica os prefixos alternativos ab e cb; q_6 e q_7 modelam o corpo intermediário (acumulação de qualquer cadeia com pelo menos um c); e os sufixos ba e cc são capturados por (q_8, q_9) e (q_{10}, q_{11}), convergindo para o estado final q_{12}.

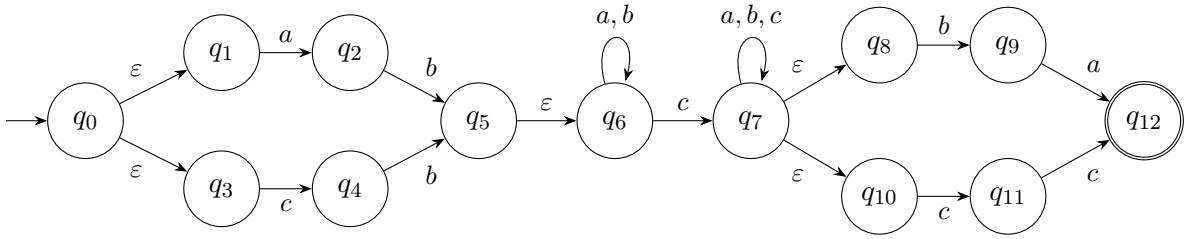


Figura 12 – NFAε `nfae_10` (13 estados, $\Sigma = \{a, b, c\}$).

A construção de subconjuntos seguida de minimização produz um DFA com 8 estados. A Tabela 2 apresenta a correspondência entre os estados renomeados (D_0–D_7) e os subconjuntos de estados do NFA, e a Tabela 3 detalha todas as transições.

Tabela 2 – Correspondência de estados — DFA mínimo de `nfae_10`.

Estado DFA	**Subconjunto NFA**	**Final?**
D_0	$\{q_0\}$	Não
D_1	$\{q_2\}$	Não
D_2	$\{q_6\}$	Não
D_3	$\{q_6, q_7, q_8, q_{10}\}$	Não
D_4	$\{q_6, q_7, q_8, q_{10}, q_{11}\}$	Não
D_5	$\{q_6, q_7, q_8, q_9, q_{10}\}$	Não
D_6	$\{q_6, q_7, q_8, q_{10}, q_{11}, q_{12}\}$	**Sim**
D_7	$\{q_6, q_7, q_8, q_{10}, q_{12}\}$	**Sim**

Tabela 3 – Tabela de transições do DFA mínimo de `nfae_10`. O símbolo "$\emptyset$" indica transição para o estado morto (eliminado pela minimização).

Estado	**Final?**	a	b	c
$\rightarrow D_0$		D_1	$\emptyset$	D_1
D_1		$\emptyset$	D_2	$\emptyset$
D_2		D_2	D_2	D_3
D_3		D_3	D_5	D_4
D_4		D_3	D_5	D_6
D_5		D_7	D_5	D_4
D_6	$*$	D_3	D_5	D_6
D_7	$*$	D_3	D_5	D_4

A Figura 13 apresenta o diagrama do DFA mínimo. Os estados D_0, D_1 e D_2 formam a cadeia de prefixo (ab ou cb); o núcleo $\{D_3, \ldots, D_7\}$ reconhece o corpo e os sufixos da linguagem, com D_6 e D_7 como estados de aceitação.

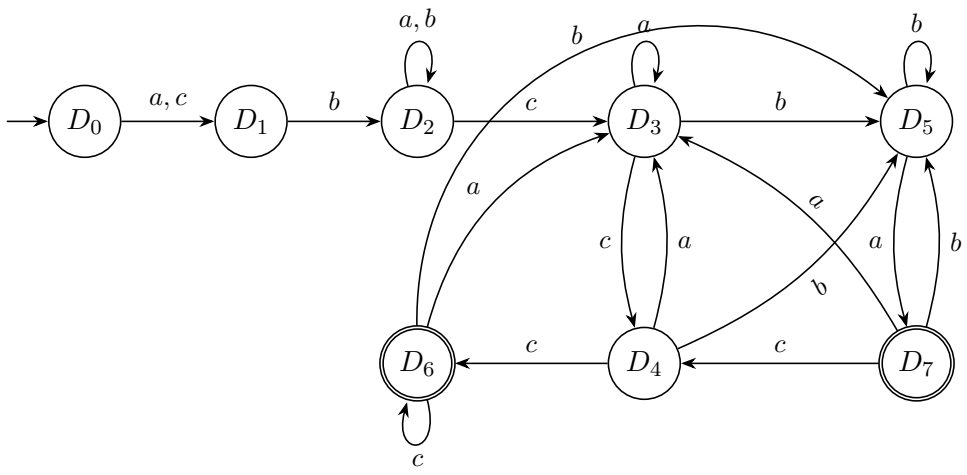


Figura 13 – DFA mínimo de `nfae_10` (8 estados, $\Sigma = \{a, b, c\}$). Estados finais: D_6 e D_7. Transições completas na Tabela 3.

3 Parte 2 — Expressões Regulares

3.1 Visão Geral — Construção de Thompson

O módulo `Regex.hs` implementa a conversão de uma expressão regular para DFA mínimo em quatro estágios:

1. **Análise sintática:** a expressão regular é convertida para uma árvore sintática abstrata (AST) por combinadores descendentes recursivos.

2. **Construção de Thompson:** a AST é percorrida recursivamente, gerando um NFAε por indução estrutural.

3. **Eliminação de ε e subconjuntos:** o NFAε é processado pelo mesmo pipeline de `Main.hs`.

4. **Minimização de Hopcroft:** o DFA resultante é minimizado.

Para cada expressão regular são apresentados a linguagem reconhecida e o DFA mínimo obtido, com renomeação dos estados por clareza.

3.2 Expressão 1 — `ab*`

A expressão `ab*` denota $L = a \cdot b^* = \{ab^n \mid n \geq 0\}$. O DFA mínimo possui 2 estados.

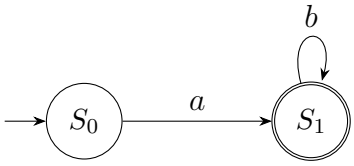


Figura 14 – DFA mínimo para `ab*`: $S_0 = \{q_0\}$, $S_1 = \{q_3, q_4, q_5\}$.

3.3 Expressão 2 — `a*b+`

A expressão `a*b+` denota $L = a^* \cdot b^+$. O DFA mínimo possui 2 estados.

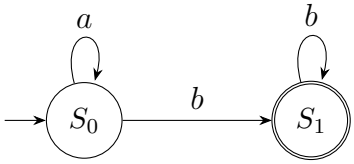


Figura 15 – DFA mínimo para `a*b+`: $S_0 = \{q_0\}$, $S_1 = \{q_7, q_8, q_9\}$.

3.4 Expressão 3 — `(a|b)*abb`

A expressão `(a|b)*abb` denota $L = (a|b)^* \cdot abb$ — cadeias sobre $\{a, b\}$ que terminam com abb. O DFA mínimo possui 4 estados.

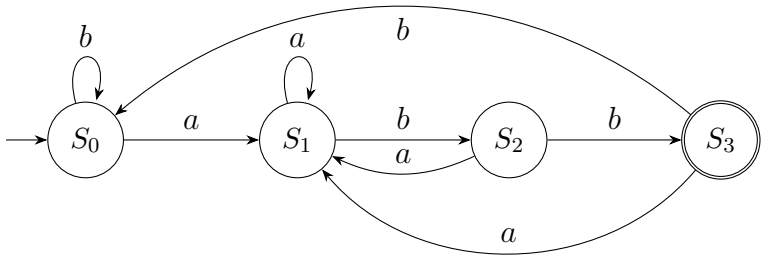


Figura 16 – DFA mínimo para `(a|b)*abb`. S_0=$\{q_0\}$; S_1=$\{q_1, \ldots, q_6, q_8, q_9, q_{10}\}$; S_2=$\{q_1, \ldots, q_4, q_6, q_7, q_8, q_{11}, q_{12}\}$; S_3=$\{q_1, \ldots, q_4, q_6, q_7, q_8, q_{13}\}^*$.

3.5 Expressão 4 — `a?`

A expressão `a?` denota $L = \{\varepsilon, a\}$. O DFA mínimo possui 2 estados, ambos finais (a cadeia vazia é aceita no estado inicial).

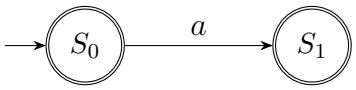


Figura 17 – DFA mínimo para `a?`: $S_0 = \{q_0\}$, $S_1 = \{q_1, q_3\}$.

3.6 Expressão 5 — `(a|b)*c(a+b|ba?)*`

A expressão `(a|b)*c(a+b|ba?)*` denota cadeias sobre $\{a, b, c\}$ compostas de: (i) qualquer prefixo em $(a|b)^*$; (ii) exatamente um c; e (iii) sufixo em $(a^+b \mid ba^?)^*$. O DFA mínimo possui 4 estados (Tabela 4).

Tabela 4 – DFA mínimo para `(a|b)*c(a+b|ba?)*`. Estados finais: R_1, R_2.

Estado	**Final?**	a	b	c
$\rightarrow R_0$		R_0	R_0	R_1
R_1	$*$	R_3	R_2	—
R_2	$*$	R_1	R_2	—
R_3		R_3	R_1	—

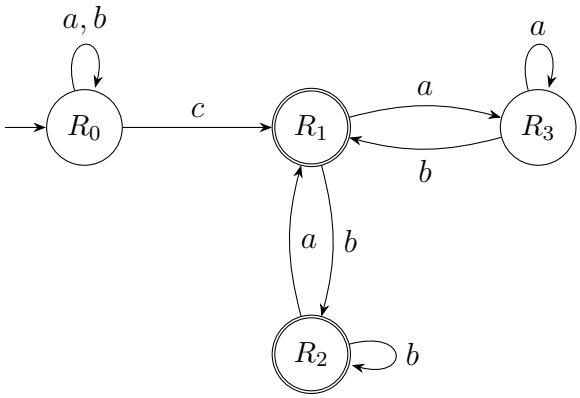


Figura 18 – DFA mínimo para `(a|b)*c(a+b|ba?)*`. R_0=$\{q_0\}$; R_1, R_2 finais; R_3 não-final.

4 Parte 3 — Atividade Complementar: Regex Crossword

4.1 Introdução

Regex Crossword é uma plataforma de quebra-cabeças interativos disponível em `https://regexcrossword.com`, na qual o objetivo é preencher uma grade com caracteres de modo que cada linha e coluna satisfaça simultaneamente a expressão regular especificada. A atividade exercita a capacidade de construir e interpretar expressões regulares, habilidade central na disciplina de Teoria da Computação.

Este capítulo apresenta as soluções para os cinco desafios da seção *Beginner* e dois quebra-cabeças originais publicados pelo aluno na seção *Player Puzzles*.

4.2 Desafios da Seção Beginner

A seção *Beginner* compreende cinco desafios introdutórios de grade 2×2. A Figura 19 apresenta os cinco concluídos.

Figura 19 – Cinco desafios da seção *Beginner* concluídos: Beatles, Naughty, Ghost, Symbolism e Airstrip One.

4.2.1 Desafio 1 — Beatles

Solução: **H**, **E**, **L**, **P** (*HELP*).

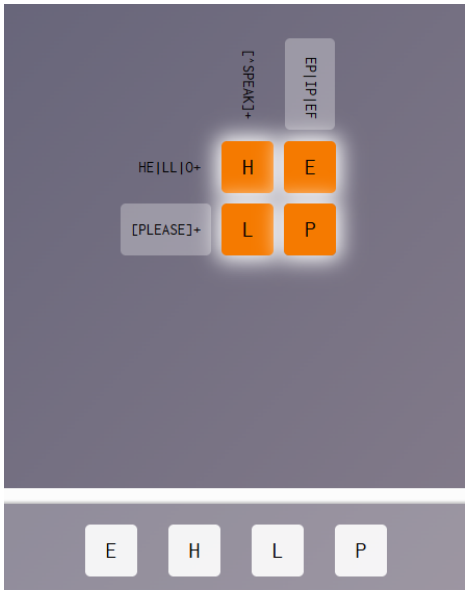


Figura 20 – Desafio *Beatles* — solução: HELP.

	HE|LL|O+	[PLEASE]+
[*SPEAK]+	**H**	**E**
EP|IP|EF	**L**	**P**

4.2.2 Desafio 2 — Naughty

Solução: **B**, **O**, **B**, **E**. O operador `\1` exige que a subcadeia repita o grupo de captura anterior.

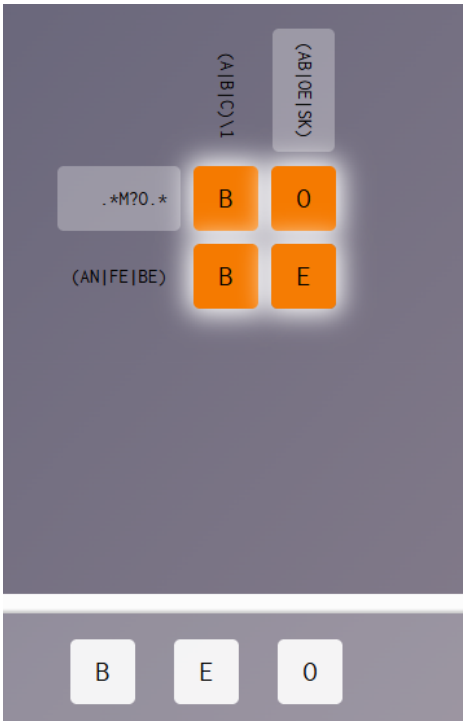


Figura 21 – Desafio *Naughty* — solução: BOBE.

	`(A|B|C)\1`	`(AB|OE|SK)`
`.*M?O.*`	**B**	**O**
`(AN|FE|BE)`	**B**	**E**

Verificação: Coluna 1 (*BB*): grupo captura `B`, referência reversa exige novo `B` ✓. Coluna 2 (*OE*): alternativa `OE` ✓. Linha 1 (*BO*): `.*M?O.*` com `M?` omitido ✓. Linha 2 (*BE*): `(AN|FE|BE)` terceira alternativa ✓.

4.2.3 Desafio 3 — Ghost

Solução: **O**, **O**, **O**, **O** (*OOOO*).

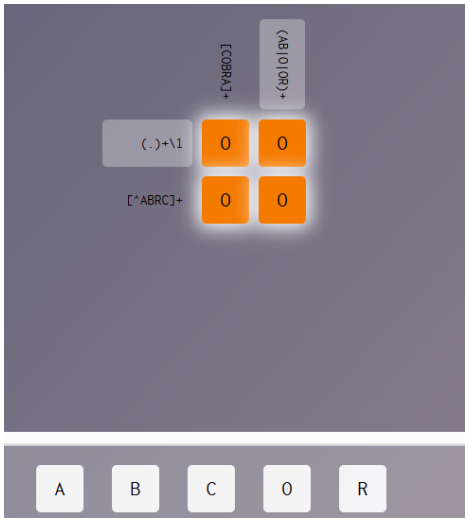


Figura 22 – Desafio *Ghost* — solução: OOOO.

	[COBRA]+	(AB|O|OR)+
(.)+\1	**O**	**O**
[^ABRC]+	**O**	**O**

4.2.4 Desafio 4 — Symbolism

Solução: *, *, /, /. Dentro de classes `[...]`, `*` e `+` são tratados como literais.

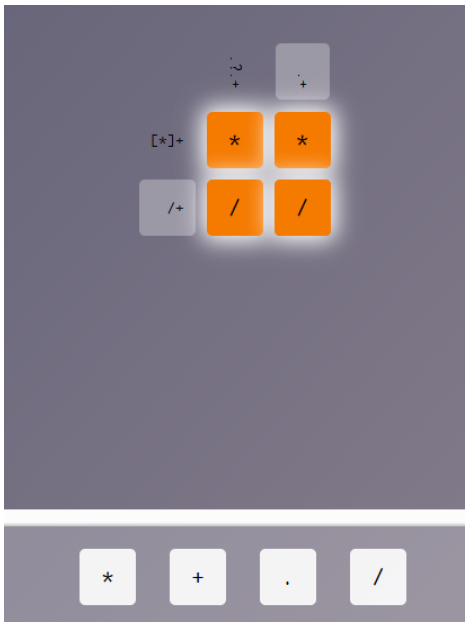


Figura 23 – Desafio *Symbolism* — solução: `**//` .

4.2.5 Desafio 5 — Airstrip One

Solução: **1**, **9**, **8**, **4** (*1984*, referência ao romance de George Orwell).

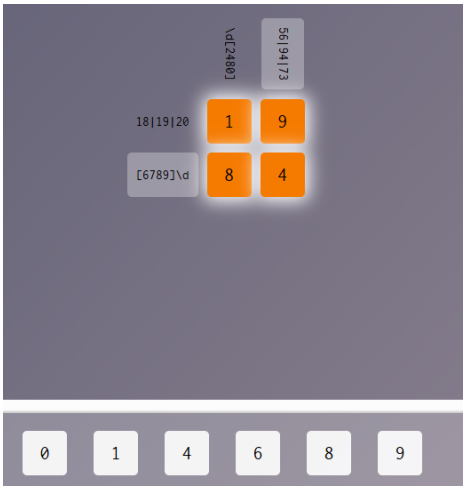


Figura 24 – Desafio *Airstrip One* — solução: 1984.

	`\d[2480]`	`56|94|73`
`18|19|20`	**1**	**9**
`[6789]\d`	**8**	**4**

Verificação: Linha 1 (*19*): `18|19|20` ✓. Linha 2 (*84*): $8 \in \{6, 7, 8, 9\}$, $4 \in \Sigma_{\text{dígitos}}$ ✓. Coluna 1 (*18*): $1 \in \Sigma_{\text{dígitos}}$, $8 \in \{2, 4, 8, 0\}$ ✓. Coluna 2 (*94*): alternativa `94` ✓.

4.3 Quebra-cabeças Criados pelo Aluno

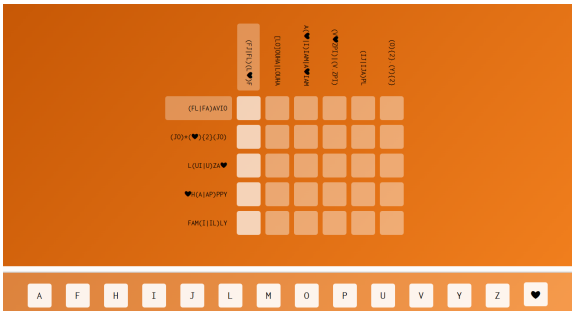
Dois quebra-cabeças originais foram criados e publicados na seção *Player Puzzles* do *Regex Crossword*. Ambos empregam referências ao nome do autor e à instituição de ensino.

4.3.1 Quebra-cabeça 1 — FLAVIO_HOUSE_IFES (Grade 5×6)

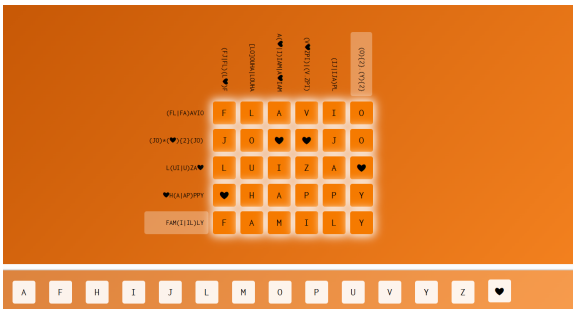
O quebra-cabeça `FLAVIO_HOUSE_IFES` é uma grade retangular de 5 linhas por 6 colunas, disponível em:

`https://regexcrossword.com/playerpuzzles/69ec93119f2fd`

As expressões regulares de linha foram projetadas para evocar palavras associadas ao nome do autor, à família e ao lar. O alfabeto utilizado inclui letras maiúsculas e o símbolo ♡ como caractere especial.



Quebra-cabeça sem solução.



Quebra-cabeça com solução preenchida.

Figura 25 – Quebra-cabeça FLAVIO_HOUSE_IFES — grade retangular 5×6.

As restrições de linha projetadas são apresentadas na Tabela 5.

Tabela 5 – Restrições de linha do quebra-cabeça FLAVIO_HOUSE_IFES.

Linha	Expressão Regular	Solução
1	`(FL|FA)AVIO`	FLAVIO
2	`(JO)+♡(2)(JO)`	JO♡JO
3	`L(UI|U)ZA♡`	LUZA♡
4	`♡H(A|AP)PPY`	♡HAPPY
5	`FAM(I|IL)Y`	FAMILY

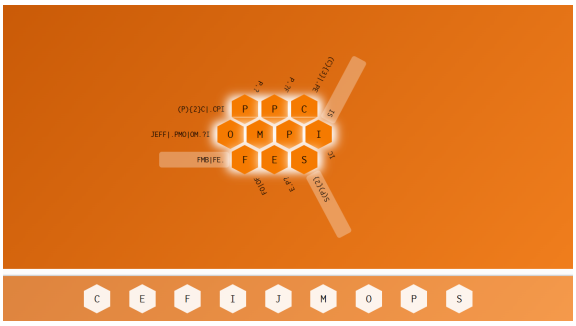
4.3.2 Quebra-cabeça 2 — FLAVIO_MB_IFES (Grade Hexagonal)

O quebra-cabeça `FLAVIO_MB_IFES` é uma grade hexagonal de tamanho 10 (arranjo 3-4-3), disponível em:

`https://regexcrossword.com/playerpuzzles/69ec8bdfb48b0`

A grade hexagonal impõe restrições em três direções — linhas horizontais e duas diagonais — o que aumenta significativamente a complexidade do design em relação a grades retangulares. O alfabeto utilizado é {C, E, F, I, J, M, O, P, S}.

Listagem do quebra-cabeça.



Quebra-cabeça com solução preenchida.

Figura 26 – Quebra-cabeça FLAVIO_MB_IFES — grade hexagonal, tamanho 10.

A solução da grade hexagonal, lida linha a linha em arranjo 3-4-3, compõe:

Linha 1:	**P P C**	(3 células)
Linha 2:	**O M P I**	(4 células)
Linha 3:	**F E S**	(3 células)

O resultado — *PPCOMPIFES* — é uma referência ao Programa de Pós-Graduação em Computação Aplicada (PPCOMPI) do Instituto Federal do Espírito Santo (IFES), programa ao qual o autor está vinculado. As restrições de linha horizontais são apresentadas na Tabela 6.

Tabela 6 – Restrições de linha do quebra-cabeça FLAVIO_MB_IFES.

Linha	**Expressão Regular**	**Solução**
1	`(P)(2)C|CPI`	PPC
2	`JEFF|PMO|OM?I`	OMPI
3	`FMB|FE.`	FES